

CCL-D: A High-Precision Diagnostic System for Slow and Hang Anomalies in Large-Scale Model Training

Yida Gu*
University of Chinese Academy of Sciences
Beijing, China
guyida21@mails.ucas.ac.cn

Fakang Wang*
Ant Group
Hangzhou, China
fakang.wfk@antgroup.com

Jianhao Fu
Ant Group
Hangzhou, China
fujianhao.fjh@antgroup.com

Zhenhang Sun
Ant Group
Hangzhou, China
zhenhang.szh@antgroup.com

Qianyu Zhang
Ant Group
Hangzhou, China
qiongyu.zqy@antgroup.com

Hairui Zhao
Jilin University
Changchun, China
zhaohr21@mails.jlu.edu.cn

Xingchen Liu
University of Chinese Academy of Sciences
Beijing, China
liuxingchen232@mails.ucas.ac.cn

Yang Tian
Ant Group
Hangzhou, China
lieyuan@antgroup.com

Wenjing Huang
University of Chinese Academy of Sciences
Beijing, China
huangwenjing23@mails.ucas.ac.cn

Zedong Liu
University of Chinese Academy of Sciences
Beijing, China
captain.liu77@gmail.com

Yifan Chen
Ant Group
Hangzhou, China
wugou.cyf@antgroup.com

Jinwu Yang
University of Chinese Academy of Sciences
Beijing, China
yangjinwu24@mails.ucas.ac.cn

Yueyuan Zhou
University of Chinese Academy of Sciences
Beijing, China
zhouyueyuan24@mails.ucas.ac.cn

Qian Zhao
Ant Group
Hangzhou, China
zq317110@antgroup.com

Haoxu Li
University of Chinese Academy of Sciences
Beijing, China
lihaoxu21@mails.ucas.ac.cn

Tao Wang†
Ant Group
Hangzhou, China
junchen.wt@antgroup.com

Feng Yu
Ant Group
Hangzhou, China
chuba.yf@antgroup.com

Zhan Wang
University of Chinese Academy of Sciences
Beijing, China
wangzhan@cqxyy.net

Guangming Tan
University of Chinese Academy of Sciences
Beijing, China
tanguangming@cqxyy.net

Dingwen Tao†
University of Chinese Academy of Sciences
Beijing, China
taodingwen@cqxyy.net

Abstract

As training scales grow, collective communication libraries (CCL) increasingly face anomalies arising from complex interactions among hardware, software, and environmental factors. These anomalies typically manifest as slow/hang communication, the most frequent and time-consuming category to diagnose. However, traditional diagnostic methods remain inaccurate and inefficient, frequently requiring hours or even days for root cause analysis. To address this,

*Yida Gu and Fakang Wang contributed equally. Gu is also affiliated with Institute of Computing Technology, Chinese Academy of Sciences.
†Corresponding authors: Dingwen Tao and Tao Wang.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PPoPP '26, Sydney, NSW, Australia
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2310-0/2026/01
https://doi.org/10.1145/3774934.3786429

we propose CCL-D, a high-precision diagnostic system designed to detect and locate slow/hang anomalies in large-scale distributed training. CCL-D integrates a rank-level real-time probe with an intelligent decision analyzer. The probe measures cross-layer anomaly metrics using a lightweight distributed tracing framework to monitor communication traffic. The analyzer performs automated anomaly detection and root-cause location, precisely identifying the faulty GPU rank. Deployed on a 4,000-GPU cluster over one year, CCL-D achieved near-complete coverage of known slow/hang anomalies and pinpointed affected ranks within 6 minutes—substantially outperforming existing solutions.

CCS Concepts: • **Hardware → Error detection and error correction**; • **Computing methodologies → Neural networks**.

Keywords: Fault tolerance, anomaly detection, diagnosic system, LLM training.

ACM Reference Format:
Yida Gu, Fakang Wang, Jianhao Fu, Zhenhang Sun, Qianyu Zhang, Hairui Zhao, Xingchen Liu, Yang Tian, Wenjing Huang, Zedong Liu, Yifan Chen, Jinwu Yang, Yueyuan Zhou, Qian Zhao, Haoxu Li, Tao Wang, Feng Yu, Zhan Wang, Guangming Tan, and Dingwen Tao. 2026. CCL-D: A High-Precision Diagnostic System for Slow and Hang Anomalies in Large-Scale Model Training. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '26), January 31 – February 4, 2026, Sydney, NSW, Australia.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3774934.3786429

1 Introduction

Deep learning is reshaping technical paradigms in critical domains, including but not limited to natural language processing [21, 49], autonomous driving [7, 58], and audiovisual technologies [25]. Guided by scaling laws, model performance has been tightly linked to the number of parameters, driving rapid expansion in model size to unlock greater capabilities [20]. However, this aggressive scaling has outpaced improvements in system reliability, exposing large-scale distributed training to increasing failure risks [52].

As model scales surpass tens of billions of parameters, ensuring training reliability has become both critical and increasingly challenging. For example, Llama3-405B experienced 419 failures over 54 days of training on 16,000 GPUs [12], yielding a Mean Time Between Failures (MTBF) of just 3 hours. The underlying causes stem from two systemic factors [9, 11, 19, 52]: (1) hardware failure rates grow linearly with device count, threatening training continuity; and (2) system complexity increases non-linearly with the scale of computation and communication. Together, these forces drive a superlinear rise in training failures, posing formidable obstacles that existing mitigation strategies struggle to overcome.

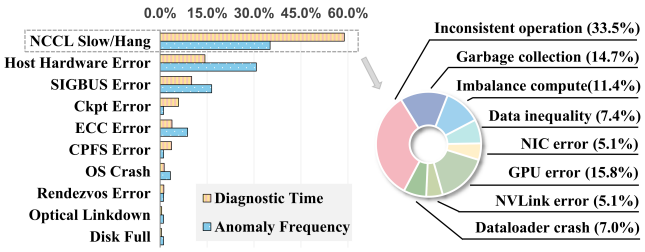


Figure 1. Training interruptions and slow/hang root-causes.

Motivation. While considerable efforts have focused on fault tolerance mechanisms—such as checkpointing [13, 24, 29, 52]—these solutions primarily aim to reduce recovery latency. However, they fall short of addressing the root causes of the widening reliability gap in large-scale training. Simply restarting failed tasks without identifying and addressing the underlying issue, such as an anomalous node, can lead to repeated failures and prolonged training delays. Therefore, diagnosing and pinpointing the causes of failures is essential to prevent recurrence, highlighting the *urgent need* for high-precision diagnostic systems in large-scale training.

Among the various functionalities of diagnostic systems for detecting training interruptions, non-obvious communication slowdowns or hangs (Slow/Hang) caused by anomalies are particularly problematic. Unlike explicit issues—such as training misconfiguration, which conventional debugging tools or log inspection typically reveal [33]—these implicit anomalies often stem from hardware, software, or environmental factors and frequently evade timely identification.

For example, Figure 1 presents a 3-month observation from our 1,000-H800 GPU heterogeneous cluster running diverse model training tasks, during which 91 training interruptions occurred. Notably, slow and hang anomalies comprised 35.2% of incidents yet consumed 58.8% (70 hours) of total diagnostic time—making them the most frequent and costly to resolve. Their diverse and complex root causes (see Figure 1, right) hinder effective diagnosis, and both their frequency and diagnostic overhead grow superlinearly with cluster size, highlighting the *critical impact* and *urgent need* for targeted mitigation measures.

Limitations of Existing Solutions. Despite the availability of several diagnostic solutions, two limitations hinder their effectiveness in addressing slow/hang. **(1) Unacceptable low diagnostic accuracy.** Common mechanisms—such as PyTorch watchdog [46], which triggers hang detection after a 30-minute timeout—offer no insights into the root cause. More importantly, most existing solutions [14, 22, 35] largely overlook slow anomalies, and the few [11, 53] that address them lack the precision needed to pinpoint the exact location. **(2) Prohibitive diagnostic overhead.** Constrained by their underlying diagnostic design, traditional methods—such as bisection-based stress testing [44] or expert-driven stack analysis [8]—often take hours to days on large GPU clusters—rendering them impractical for real-time mitigation.

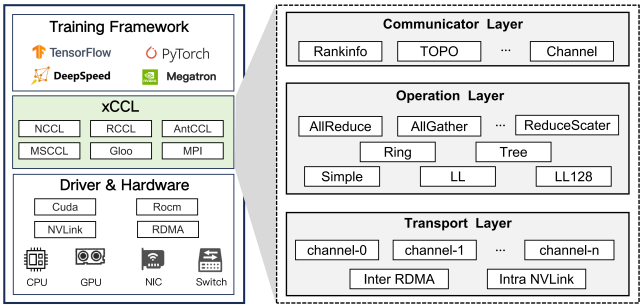


Figure 2. Position of CCL in training and its hierarchical structure.

These gaps in diagnostic accuracy and efficiency have become a *critical bottleneck* in advancing the reliability of large-scale distributed training systems.

Our Solution. To address these challenges, we propose **CCL-D** , a diagnostic system capable of automatically detecting and precisely locating slow/hang anomalies within minutes. The core idea behind CCL-D is to extend the functionality of CCL by enabling fine-grained identification and tracing of communication traffic. Rather than identifying the devices (e.g., switches or routers) that cause the anomalies, CCL-D aims to pinpoint the affected GPU(s) within the node topology and provide a probable root-cause analysis. To the best of our knowledge, *CCL-D is the first diagnostic system to leverage both host-level and GPU kernel-level communication states for high-precision slow/hang diagnosis.*

Our main contributions are summarized as follows:

- We conduct a comprehensive analysis of collective communication workflows and slow/hang anomalies in different model training scenarios and, for the first time, derive six categories based on their potential root causes.
- We design a high-precision slow/hang diagnostic technique by extending CCL with a set of portable, cross-layer metrics centered on Send/Recv primitives. Integrated with a precision decision analysis algorithm, this design enables timely and accurate anomalies detection and location.
- We implement a lightweight distributed tracing framework that accurately captures and measures per-round communication traffic with negligible overhead (<1%), enabling scalable deployment on production clusters.
- We validate CCL-D's functionality and scalability by training large models on a 4,000-GPU cluster. Compared to the hours or even days required for model training, CCL-D diagnoses slow/hang anomalies with high precision within 6 minutes, significantly outperforming existing methods.

2 Motivation and Challenges

2.1 Workflows of CCL Primitive Operations

A single GPU can no longer meet the storage requirements of large models. To address this, distributed training frameworks utilize strategies such as DP [23, 40], TP [50], and PP [16, 32, 56, 57] to partition data and models, with each GPU storing a subset. This leads to the fact that training

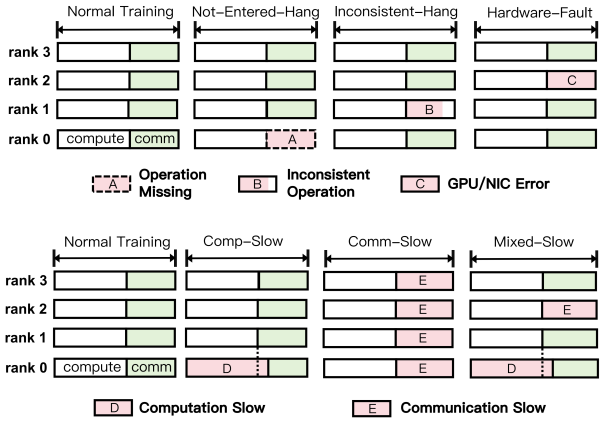


Figure 3. Hang (upper) and Slow (lower) scenarios analysis.

relies on collective communications to synchronize model parameters across GPUs. Therefore, as shown in Figure 2, the stability of CCL, which bridges high-level model services with low-level computational resources, is critical.

In large-scale model training, different parallel strategies invoke various collective communication operations such as AllReduce, AllGather, ReduceScatter, and AlltoAll. Although their specific functions differ, these workflows can generally be abstracted into three critical phases: 1) Domain initialization: the host identifies participating ranks (i.e., GPUs) and establishes a communication domain, that is, the communicator; 2) Kernel dispatch: the CPU dispatching communication kernels (e.g., AllReduce) to the corresponding GPUs for execution; 3) Concurrent data transfer: multiple channels transfer data concurrently using intra-node NVLink [5] or inter-node RDMA [39] hardware within kernels. Collective communications require all ranks within a communicator to behave consistently, including but not limited to algorithms (e.g., Ring, Tree [4, 15]), protocols (e.g., Simple, LL, LL128 [34]), and communication operation (i.e., GPU kernel) counts. Any deviation may lead to CCL slow/hang.

2.2 Slow/Hang Anomalies Classification

Coarse-grained classifications such as "Slow" or "Hang" are insufficient for timely root cause identification and often delay remediation. Therefore, through analyzing slow/hang anomalies observed over the past 2 years in our production clusters and related reports in prior studies, we found that most issues are closely tied to communication workflows. Based on the analysis of Section 2.1, we derive six fine-grained anomalies' root causes (Figure 3) that comprehensively cover slow/hang patterns in modern CCL systems:

Hang Anomalies—**(H1) Not-Entered-Hang:** Some ranks in the communicator miss a communication operation and could not enter collective communication; **(H2) Inconsistent-Hang:** Inconsistent communication operations performed by individual GPUs within a communicator at the same time, leading to conflicts; and **(H3) Hardware-Fault:** Hardware

(eg., GPU, NIC) or driver faults occurring on individual device during collective communication.

Slow Anomalies—**(S1) Computation-Slow:** Existing ranks enter communication later due to prolonged pre-computation, slower data loading, or GPU frequency throttling induced by thermal constraints, slowing down the communication; **(S2) Communication-Slow:** Real-time network fluctuations or congestion can lead to performance degradation; and **(S3) Mixed-Slow:** Scenarios where both computation slow and communication slow occur simultaneously.

Over the past year, we have observed that the ratio of hang to slow cases was 62.1% and 37.9%, respectively. For hang, Not-Entered-Hang accounted for 11.8%, Inconsistent-Hang for 58.9%, and Hardware-Fault for 29.3%. For slow, comp-slow represented 81.8%, comm-slow 11.1%, and mix-slow 7.1%. Given the stability of collective communication workflows and hardware architectures in the near future, the proposed taxonomy remains broadly applicable and can be further refined for deployment-specific needs.

2.3 Prior Diagnostic Works and Their Limitations

Although several existing methods offer slow/hang anomaly detection or location, they each has specific limitations.

Bisection-based Methods are offline and reactive, relying on anomalies to be first detected manually before location can begin. When locating anomalies, tools like NCCL-tests [36] are used to stress-test the involved machines and iteratively isolate faults. Training logs from OPT-175B [31] and BLOOM-176B [3] show reliance on this method. However, due to the lack of runtime communication states, these methods only work for network- or hardware-related failures, and cannot reproduce logic-level or intermittent issues. Moreover, they are also time-consuming and resource-intensive, requiring task suspension during diagnosis.

Stack Analysis identifies anomalies by comparing function call information across all ranks. Although it can capture certain communication calls, stack traces are often verbose and complex, making their interpretation difficult and highly dependent on domain expertise and manual effort. In large-scale trainings, engineers must sift through the massive stack data generated by both CCL and the training components to make informed judgments. For example, ParaStack [22] detects hangs by randomly sampling stack traces from selected communication processes and analyzing the specific states of MPI processes. Nevertheless, stack traces may fail to capture all anomaly types, particularly those involving loops or hardware slowdowns, leading to misdiagnosis.

CCL Runtime Analysis focuses on runtime communication states, traces behaviors of the collective communication during training. NCCL RAS [35] maintains a thread per rank to exchange state and monitor inconsistencies, but only offers coarse-grained information, such as current rank status and communication operation invocation counts, limiting its diagnostic resolution. Alibaba C4D [11] extends this approach with additional metrics (e.g., data transfer time, receiver wait time), enabling basic detection but low location precision of slow anomalies. Greyhound [53] intercepts communication calls to monitor iteration time during training, detect slow anomalies only, halt training upon detection, and locate slow rank through stress testing.

Unlike the first two categories, CCL-D represents the third class of approaches, diagnosing anomalies from CCL-level runtime states. By introducing fine-grained kernel metrics beyond host-level information, CCL-D achieves high-precision coverage of slow/hang anomalies with minimal overhead, while ensuring training continues uninterrupted.

2.4 Our Intuition and Research Challenges

Any slow/hang anomaly can be diagnosed by analyzing the CCL states, which ultimately manifest as behavioral discrepancies among the participating ranks. This calls for finer-grained, multi-dimensional kernel-level metrics to more precisely capture communication behaviors and pinpoint anomalies. Nevertheless, two key challenges must be addressed:

Challenge 1: Effective design of kernel metrics capable of accurately characterizing and diagnosing slow/hang anomalies remains a non-trivial task. Although existing profiling tools (e.g., NVIDIA NCU [37]) expose hundreds of runtime metrics (e.g., register utilization, SM occupancy), these signals are primarily designed to reflect the kernels' computation states rather than capturing the interactive nature of collective communication. Hence, they fail to reveal the root causes of slow/hang behaviors. Furthermore, the complex many-to-many communication patterns and deeply nested loops inherent in collective operations make custom measurement logic, such as embedding timers, highly intrusive, requiring careful consideration of synchronization overhead and often altering kernel behavior. This results in high maintenance costs and poor portability. Therefore, it is crucial to identify kernel-level information tightly coupled with slow/hang anomalies and to construct a minimal, complementary subset to augment existing metrics.

Challenge 2: Achieving high-precision communication traffic measurement and analysis under low overhead is a key challenge. In large-scale training, hybrid parallelism introduces frequent and highly concurrent collective operations, with overlapping communicators that exacerbate measurement complexity. A naive solution, centralized registration and unified traffic management, would incur excessive synchronization delays and high-frequency data accesses, becoming itself a scalability bottleneck. In addition, although directly measuring and storing kernel-level metrics on each rank is simpler to design, it would consume valuable GPU compute and memory resources, interfering with model training. Therefore, to prevent the diagnostic system from becoming a performance bottleneck, a lightweight and scalable metric tracking mechanism is essential.

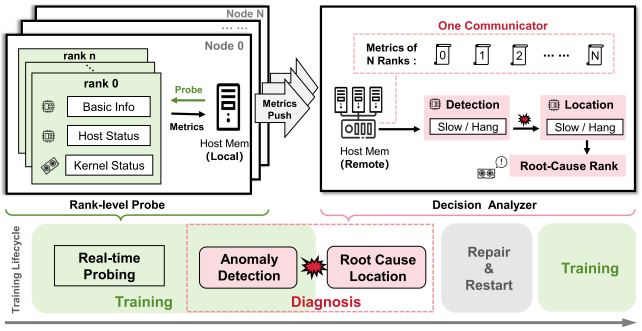


Figure 4. CCL-D and training lifecycle under anomalies.

3 Overview of CCL-D

To address these challenges, we propose CCL-D, a diagnostic framework for rapid detection and location of slow/hang anomalies. As illustrated in Figure 4, CCL-D consists of two core modules: a distributed real-time rank probing module and a centralized decision analysis module.

The **Rank Probing Module** is deployed on each participating rank. It collects not only basic rank states but also a carefully designed set of multi-level metrics spanning both host and GPU kernel states to comprehensively characterize slow/hang anomalies (Section 4.1). *Our design aims to minimize the number of metrics while maximizing their diagnostic coverage and utility, thereby reducing probing complexity.* A lightweight distributed tracing mechanism is implemented using basic rank information, and kernel metrics are measured with minimal overhead by leveraging host-side resources only (Section 5).

The **Decision Analysis Module** is a centralized but scalable component shared across ranks (Section 4.2). It periodically processes metrics from all ranks in a communicator to detect slow/hang anomalies, and upon detection, applies specialized decision algorithms to accurately locate the root-cause ranks. Unlike a single-node design, this module operates as a small distributed cluster for improved scalability.

Importantly, the decision analysis operates out-of-band, decoupling metric analysis from training execution to avoid any impact on training performance or accuracy. While online metric collection incurs a slight runtime overhead (<1%), this impact is negligible compared to the significant reduction in the overhead of anomaly detection and location.

4 Design of Metrics and Decision Analyzer

Before discussing metric measurement, this section first introduces the design of metrics, followed by how the decision analyzer leverages them for anomaly detection and location.

4.1 Cross-layer Probing Metrics Design

We design metrics across three layers of the CCL stack, providing multi-perspective coverage of communication behavior. Figure 5 shows the mapping between these metrics and the anomaly scenarios they target. The metrics in the basic

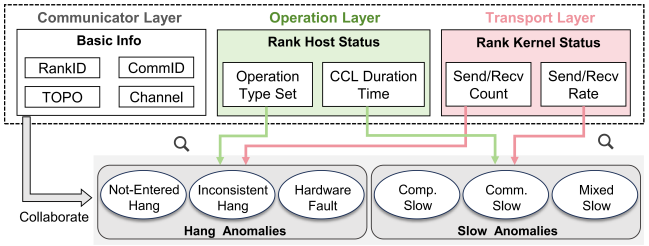


Figure 5. Metrics of CCL-D and corresponding anomaly types.

information layer serve both as a means for communication traffic identification and as a foundation for basic diagnosis.

4.1.1 Analysis of Kernel-level Metrics.

To improve diagnostic accuracy while minimizing system complexity, it is essential to identify metrics that capture the root causes of Hang/Slow anomalies, thereby reducing reliance on redundant kernel information. In the context of collective communication, which fundamentally consists of data exchange between GPUs, the atomic Send and Recv operations naturally meet these criteria. As the foundational primitives of all collective operations, they are independent of underlying hardware topologies, communication protocols, or CCL implementations, ensuring strong generality and portability. Empirical analysis further shows that in a correctly completed collective operation, all participating ranks exhibit consistent Send/Recv behaviors, demonstrating high comparability and structural regularity.

In contrast to methods that only monitor kernel invocations in the operation layer, Send/Recv activities provide fine-grained visibility into the transport layer and can expose low-level issues invisible at the host level. Moreover, by focusing solely on Send/Recv behavior, they introduce minimal overhead, require only lightweight kernel modifications, and are well-suited for scalable and efficient deployment.

4.1.2 Metrics-Based Characterization of Slow/Hang.

To tightly couple with the root causes of slow/hang, we design two types of kernel-level metrics based on Send/Recv behavior: one measuring count, the other rate.

Hang Diagnostic Metrics. Hang anomalies are fundamentally characterized by imbalanced Send/Recv counts across ranks. To capture this, we introduce ***SendCount*** and ***RecvCount***, which record the actual number of send and receive instructions executed within the kernel. Compared to operation-layer invocation counts, these metrics can detect cases where operation counts are consistent but underlying Send/Recv behaviors diverge. To further support the detection of inconsistencies caused by mismatched communicator or scheduling errors, we also introduce a host-level metric—***Operation Type Set***—that records static metadata for each rank, including the communication algorithm, protocol, data size, and operation name. These parameters remain constant throughout the entire communication.

Slow Diagnostic Metrics. Slow anomalies manifest as degraded Send/Recv rates. While operator-level timestamps

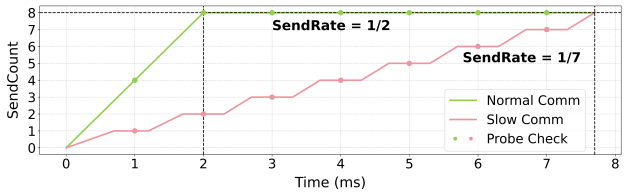


Figure 6. Comparison of SendRate between normal and slow ranks.

can be used to estimate kernel ***Duration Time*** per rank to make an initial judgment, this approach is limited by NTP clock drift [6, 28] and the millisecond-to-microsecond scale of collective operations, making the time-based diagnosis inaccurate. Moreover, SendCount/RecvCount fail to capture such anomalies, as final counts may remain consistent despite underlying performance degradation.

To overcome this, we model communication progress as a cumulative count function $C(t)$. Under normal conditions, the derivative dC/dt is nearly constant, reflecting a stable communication rate; during a slow anomaly, it drops markedly. We define ***SendRate*** and ***RecvRate*** as approximations of dC/dt, computed as the reciprocal of the number of changes in SendCount/RecvCount within a fixed sampling window. This design captures fine-grained rate variations without relying on global clock synchronization. Figure 6 illustrates a specific use case, where the value change is probed every 1 ms. In normal communication, 8 send operations complete with 2 value changes, giving a rate of 1/2; under a slow anomaly, the same operation takes 7 changes, dropping the rate to 1/7 and clearly exposing the degradation.

4.2 Architecture of Decision Analyzer

Cross-layer metrics provide critical signals for diagnosing slow/hang anomalies but cannot alone determine their occurrence or pinpoint root causes. To address this, CCL-D employs a centralized decision analyzer that systematically aggregates metrics from all ranks for anomaly detection and root-cause location.

4.2.1 Slow/Hang Automatic Detection. To enable accurate anomaly detection, CCL-D uses the communicator ID to group metrics and applies specialized rules for analysis. Furthermore, our analyzer must distinguish between Ring and Tree communication algorithms to ensure correct grouping of comparison targets within the same communicator. While we do not focus on CCL-specific implementation details, we leverage the fundamental design principles of Ring and Tree to guide this grouping. In Ring, all ranks form a closed loop with equal in-degree and out-degree, enabling uniform analysis across the entire communicator. In contrast, Tree has a hierarchical structure in which only ranks within the same tree layer exhibit consistent SendCount/RecvCount and update frequencies; diagnostics are therefore restricted to same-layer ranks under tree topology.

Hang anomaly detection relies on the dimension of time. When the communication duration time of a rank exceeds the time threshold, our decision analyzer will trigger the creation of a hang anomaly alert. However, in large-scale model training, long-duration operations such as checkpoint or synchronization operations may occur normally. To reduce false positives, we set the hang threshold considering such long operations and further filter out alerts where the operation is *AllReduce* with communication data size less than 4 Bytes (i.e., a barrier).

Slow anomaly detection is more nuanced and consists of two categories: *slow-at-start* and *in-communication slowdown*, each requiring a different temporal baseline. In the early training phase, when no historical statistics are available, the administrator specifies an initial baseline $T_{\text{base}}^{(\text{init})}$ based on prior experience with similarly scaled models. The analyzer then dynamically updates this baseline by averaging the maximum durations of the first m communication rounds, where m is the smaller of 100 rounds or all rounds within the first two minutes, yielding $T_{\text{base}}^{(\text{new})}$. Formally:

$$T_{\text{base}} = \begin{cases} T_{\text{base}}^{(\text{init})}, & \text{if } r \le m, \\ \frac{1}{m}\sum_{j=1}^{m} T_{\text{max}}^{(j)}, & \text{otherwise} \end{cases} \tag{1}$$

where r is the current number of communication rounds, and $T_{max}^{(j)}$ denotes the maximum duration time among all ranks participating in round j.

To capture representative slowdowns while avoiding the overhead of per-round analysis, we operate on a fixed one-minute detection window. Within each interval, the intra-round time range $\left(T_{\text{max}}^{(r)} - T_{\text{min}}^{(r)}\right)$ is computed for every round, and the round with the largest range is selected. Its maximum communication duration is defined as T_{max}, focusing detection on the round with the most pronounced inter-rank disparity and avoiding misjudgments from relying solely on the absolute longest communication time. Formally:

$$T_{\text{max}} = T_{\text{max}}^{\left(\arg\max_{r \in \text{window}} \left[T_{\text{max}}^{(r)} - T_{\text{min}}^{(r)}\right]\right)} \tag{2}$$

Finally, the slowdown ratio is:

$$R = (T_{\text{max}} - T_{\text{base}})/T_{\text{base}} \tag{3}$$

A slow anomaly is flagged when $R > \theta_{\text{slow}}$. Instead of using a fixed subjective threshold, we determine θ_{slow} statistically, and in practice the threshold is usually close to 3. To further reduce false positives, cases involving barrier synchronization are excluded, and transient slowdowns caused by cluster jitter are ignored unless they recur. For this purpose, CCL-D maintains a cumulative counter of slow detections and triggers location only when the repetition threshold is exceeded.

4.2.2 High-Precision Root Cause Location. Although detecting slow/hang can confirm anomalies within a communicator, it cannot directly identify the rank that triggered

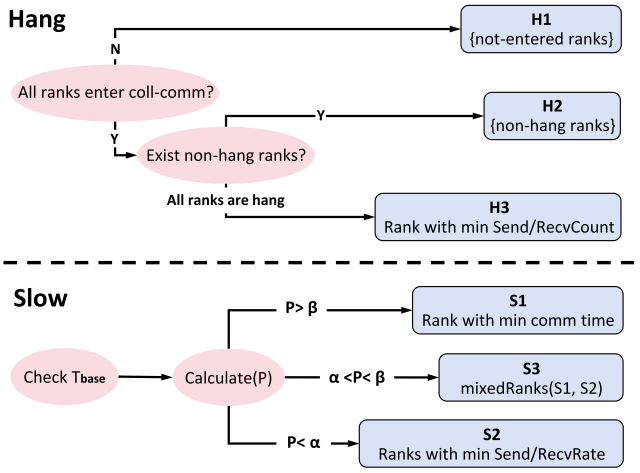


Figure 7. Decision tree of root-cause ranks. **H1-H3** and **S1-S3** correspond to the hang/slow discussed in Section 2.2.

them. In large-scale clusters, anomalies may propagate across ranks, making the true root-cause rank difficult to isolate. To address this, the location module analyzes kernel metrics from all ranks and applies distinct algorithms to separate root-cause from secondary effects. Figure 7 illustrates the design logic. As the decision rules only compare metrics across participators, the algorithm runs in O(N) time for N ranks, remaining scalable to thousands of GPUs.

Hang anomaly location is formalized as a pure classification problem over rank states, using Trace ID as the first indicator (details in Section 5.1). A rank with no increment in its operation counter is inferred not to have entered the collective, and is identified as the root cause of a Not-Entered Hang. When all ranks have entered, two scenarios remain: One is the presence of non-hang ranks, which can be attributed to inconsistent operations, and the non-hang ranks are the root cause. The other is that all ranks are hang, suggesting a problem with hardware. The rank with the fewest SendCount/RecvCount is considered the root-cause rank.

Slow anomaly location begins by checking whether the current T_{base} is a configured value. A static baseline suggests a slow-at-start scenario, while a dynamically updated baseline indicates in-communication slowdown, guiding subsequent handling strategies accordingly. We then introduce P to formalize the relative contribution of computation delay versus communication delay in the current round:

$$P = (T_{max} - T_{min})/(T_{max} - T_{base}) \tag{4}$$

T_{min} and T_{max} are similar in definition, except that the former takes the minimum communication time.

In Equation (4), T_{min} serves as a sliding indicator within the range $[T_{base}, T_{max}]$. When computation is the bottleneck, the last rank entering communication drives T_{min} close to T_{base}, making the numerator and denominator nearly equal and pushing $P \longrightarrow 1$. Conversely, when communication dominates, T_{min} approaches T_{max}, diverging from T_{base} and driving $P \longrightarrow 0$. Conceptually, $P = 0$ corresponds to purely slow communication, $P = 1$ to purely slow computation, and $P = 0.5$ to an equal contribution of both. However, in large-scale clusters, the equal-contribution scenario rarely occurs. To emphasize the dominant anomaly type, we introduce two boundary parameters α and β around 0.5 (e.g., $\alpha = 0.4$, $\beta = 0.6$). When $P > \beta$, the anomaly converges to computation-slow, and the rank with minimal communication time is designated as the root cause; when $P < \alpha$, it is attributed to communication-slow, with the minimal SendRate/RecvRate rank as the root cause. Values in between suggest a mixed slowdown, requiring analysis of both slow types. This formalization provides a principled basis for rank attribution.

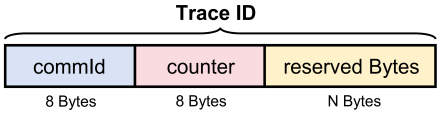


Figure 8. The structure of Trace ID.

5 Metric-Aware Tracing Framework

Large-scale high-precision diagnostics face critical overhead challenges spanning metrics probing, storage, and submission. Unoptimized overhead impedes system scalability and hinders thousand-GPU model training. To overcome this, we design a lightweight distributed tracing framework that achieves precise communication traffic attribution and low-overhead metrics measurement via decentralized identification and host-driven measurement workflow.

5.1 Distributed Communication Tracing

To achieve precise and scalable recognition of concurrent communications, we introduce two complementary data structures: the Trace ID at the communicator level and the Probing Frame at the rank level.

Trace ID. This structure provides a decentralized mechanism to uniquely label each round of communication. As shown in Figure 8, Trace ID consists of the communicator ID, an communication operation counter, and an optional extension field. Since all participating ranks increment their local counters synchronously at the start of each round, Trace ID ensures consistent operation labeling across the communicator. The extension field accommodates timestamps or status flags, supporting fine-grained traceability.

Probing Frame. Each rank maintains a Probing Frame to capture runtime states of communication kernels. As shown in Figure 9, structurally consisting of a header and a body, it provides a compact and reusable mechanism for recording kernel-specific metrics. Since GPU kernels execute first-in-first-out (FIFO), we can avoid frequent memory allocation overhead by reusing the header and body. Hence, only one probing frame per rank is sufficient.

The header uniquely identifies each communication kernel and contains four fields: the operation counter, a mode

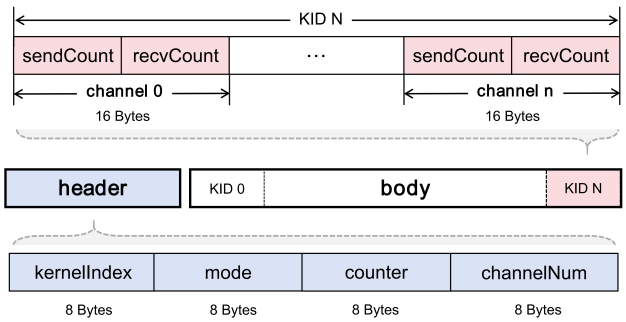


Figure 9. The structure of Probing Frame.

flag (indicating whether metric measurement is enabled), the kernelIndex, and the number of communication channels. The body is cyclically partitioned into blocks, with kernelIndex specifying the block position for the current operation (computed as counter modulo the number of blocks). Within each round of communication, every channel is assigned two consecutive slots to record the SendCount and RecvCount. The number of channels that is correlated to the number of NICs and established during the CCL initialization. By separating records across channels, this design not only supports fine-grained diagnostic but also distinguishes intra-node RDMA anomalies from inter-node NVLink anomalies, since a rank may employ different channels in successive phases.

Together, these two diagnostic data structures enable lightweight multilevel tracing and storage of collective communication states, ensuring both consistency and scalability of cross-layer diagnostic information.

5.2 Host-Driven Metrics Measurement

Directly measuring metrics on GPUs not only risks memory overflow, but also interferes with normal training execution. To avoid this, the measurement process must remain non-blocking and non-intrusive to GPU kernels. Our design principle is therefore to minimize GPU memory and compute overhead while preserving diagnostic accuracy. We further observe that host resources are typically underutilized during training, consistent with prior studies [17, 18, 51]. Guided by this insight, CCL-D shifts most of the metric measurement and computation to the CPU and host memory.

Specifically, CCL-D employs a zero-copy memory sharing strategy based on CUDA Unified Virtual Addressing (UVA), enabling implicit DMA propagation of metrics without explicit synchronization. This contiguous pinned memory shared between GPU and CPU stores the probing frames of all local ranks, allowing high-frequency metric updates during Send/Recv operations (① in Figure 10). To further avoid occupying GPU computational resources, a CPU diagnostic thread periodically reads the pinned memory to perform lightweight computations, such as updating kernels' SendRate/RecvRate from their SendCount/RecvCount changes (②). Upon kernel completion, a callback function is invoked (③), which notifies the diagnostic thread to push

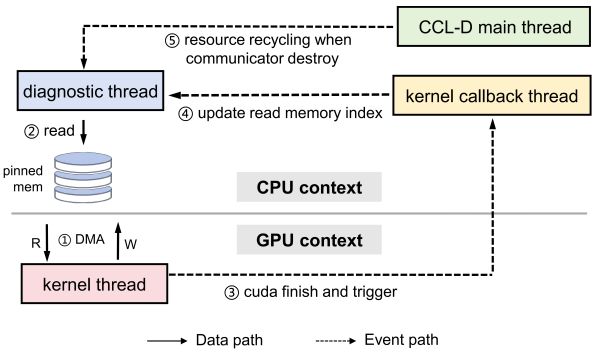


Figure 10. Metrics measurement workflow.

current metrics to the decision analyzer and advance the buffer pointer to the next block (④). Finally, once the communicator is destroyed, associated diagnostic resources are released promptly (⑤), ensuring minimal overhead on host.

6 Evaluation

6.1 Evaluation Setup

Platform: We evaluate CCL-D on a GPU cluster where each node is equipped with 2 Intel 8469C CPUs, 800 GB RAM, and 8 Nvidia H20 GPUs with 96 GB HBM3 each, interconnected via 900 GB/s NVLink. The nodes are further connected through 4 ConnectX-7 400G NICs. All nodes have the same software, including CUDA 12.2, NCCL 2.24.3, Pytorch 2.4.0, and Megatron 0.9.0. For functional verification, we use two nodes with 16 GPUs. To assess scalability, we expand the experiments up to 4,000 GPUs. To show the generality of CCL-D, we also deploy and evaluate its ROCm-based counterpart, RCCL [2].

Baselines: We select 5 approaches detailed in Section 2.3. The bisection method works by using a divide-and-conquer approach and stress-testing with NCCL-tests. We use DLRover [44] as the specific implementation for bisection. For stack analysis, we develop our own baseline by integrating the design principles of XPUTimer [8] and ParaStack. For CCL runtime analysis, we use NCCL RAS [35], Greyhound [53], and C4D [11] as baselines for evaluation.

Parameters: For fairness, both C4D and CCL-D use a 5-minute hang threshold and a 1-minute slow-detection window. 5-minute bound reflects checkpoint and normal barrier durations, and our cluster statistics show that 97% of cases exceeding this limit cannot recover, making it a reliable cutoff. 1-minute window not only complies with cluster Service Level Objectives (SLO), but also captures sufficient collective communication rounds to avoid per-round diagnosis overhead, thus balancing sensitivity and diagnostic.

Workloads: We use 2 communication algorithms, Ring-based and Tree-based [4, 15], and consider 3 communication protocols, Simple, LL and LL128 [34], to test the accuracy and efficiency of CCL-D. BaiLing-5B, Llama2-7B [47], Llama3.1-8B [12] and BaiLing-80B [45] are used as training tasks to measure and analyze the various overheads of CCL-D. We

Table 1. Comparison of diagnosis accuracy and efficiency across baseline methods and CCL-D.

Diagnosis Method	Diagnosis Accuracy						Diagnosis Efficiency				Runtime Dependency
	Not-Entered Hang	Inconsistent Hang	Hardware Fault	Comp. Slow	Comm. Slow	Mixed Slow	Hang Detect	Slow Detect	Hang Locate	Slow Locate	
Bisection	✗	✗	✓	✗	✓✗	✗	> 30 min	> 1 h	4 min	4 min	Offline
Stack	✓	✓	✓	✗	✗	✗	> 30 min	N/A	5 min	N/A	Online
RAS	✓	✗	✗	✗	✗	✗	> 30 min	N/A	10 ms	N/A	Online
Greyhound	✗	✗	✗	✓✗	✓	✗	N/A	1min	N/A	1.43 s	Online + Offline
C4D	✓	✗	✗	✗	✓	✗	5 min	1min	104 ms	138 ms	Online
CCL-D	✓	✓	✓	✓	✓	✓	**5 min**	**1 min**	**108 ms**	**146 ms**	**Online**

• In all experiments, the communication algorithms (Ring, Tree) and protocols (Simple, LL, LL128) did not affect the results.
• N/A: The method does not support this anomaly diagnosis.

use Alpaca [43] and Fineweb-edu [38] datasets to train them with two training strategies, FSDP [59] and 3D [41] parallel.

6.2 Performance of Anomaly Diagnostic

6.2.1 Diagnostic Accuracy and Efficiency. An effective diagnostic system must balance both accuracy and efficiency. Table 1 compares six diagnostic approaches from these two perspectives. This experiment is conducted on 16 devices. To comprehensively evaluate the detection and location capabilities, we construct diverse anomaly scenarios. For hang, cases include process blocking (e.g., SIGSTOP/SIGCONT), inconsistent operations, and NIC/GPU failures. For slow, we simulate computational bottlenecks (e.g., GPU frequency throttling, garbage collection interference), communication delays (e.g., link jitter, network misconfiguration, artificial delays), and system-level resource contention. This coverage ensures a thorough assessment of the diagnostic system's effectiveness.

Diagnostic Accuracy. It encompasses two aspects: *coverage of anomaly scenarios and precision of faulty-rank location.* By leveraging multi-dimensional metrics such as send/recv counts and rates, CCL-D achieves comprehensive coverage of both slow/hang anomalies with precise rank-level attribution. For RCCL diagnostic, CCL-D achieves the same diagnostic accuracy as NCCL scenarios.

In contrast, baseline methods show clear limitations. Bisection cannot reproduce real-time slow/hang and is restricted to hardware or network faults, requiring long testing cycles for location. Stack analysis covers all hang anomalies but fails to detect slow ones, since stack states remain identical across ranks. RAS only records host-level operation counts, which makes it incapable of handling slow anomalies and occasionally producing false positives in our experiments. Greyhound lacks mechanisms for hang; for comp-slow, it only captures computation delays caused by GPU frequency throttling, while failing to account for algorithm-level factors such as garbage collection. C4D lacks kernel-level insight and only relies on coarse-grained timing information, which makes it can only identify comm-slow in practice. Notably, both C4D and Greyhound fail to handle mixed-slow scenarios, leading to reduced location accuracy.

Table 2. Comparing slow/hang diagnostic efficiency in one-month periods before and after CCL-D integration **(GPU scale: 16-4000)**.

Type	Diagnosis Mode	Cases Captured	Avg. Diagnosis Time	Root Causes
Hang	Manual	4	47 h	NIC(H3), GPU(H3), UN
	CCL-D	**18**	**6 min (5+1)**	IO(H2), GPU(H3) NV(H3), NE(H1)
Slow	Manual	4	74 h	UN
	CCL-D	**10**	**2 min (1+1)**	DI(S1), GC(S1) IC(S1), MS(S3)

• **Abbreviations:** UN=Unknown; NIC/GPU=NIC/GPU error; NV=NVLink error; IO=Inconsistent operation; NE=Not entered; DI=Data inequality; GC=Garbage Collection; IC=Imbalance Compute; MS=Mixed-Slow.

Diagnostic Efficiency. We evaluate efficiency in terms of anomaly detection latency and location latency, while also considering the impact of communication algorithms, protocols, and parallel strategies. Each test is repeated five times, and the average result is reported.

Bisection, stack analysis, and RAS do not automatically detect anomalies; instead, they rely on manual perception and intervention, making detection times highly variable. For fairness, we assume they operate under PyTorch Watchdog's default 30-minute timeout, resulting in average hang detection delays of no less than 30 minutes. Based on production statistics, users typically perceive slow anomalies only after about 1 hour, which we adopt as the baseline for slow detection by bisection. Our experiments also confirm that communication algorithms, protocols, and parallel modes have negligible influence on CCL-D's detection efficiency.

Regarding location efficiency, bisection requires multiple group tests to re-collect runtime information, while stack analysis depends on expert knowledge to compare stack differences—both inefficient, exceeding 4 minutes. RAS achieves lower latency, but since it relies solely on host-level operation counts, it can only identify Not-Entered-Hang, sacrificing both accuracy and coverage. Greyhound halts training and relies on stress testing, yielding lower efficiency than C4D and CCL-D, with second-level latency compared to their millisecond-level performance. Ultimately, CCL-D precisely locates slow/hang anomalies within 150 ms under NCCL/RCCL, which is well suited for production-scale training.

6.2.2 Scalability of CCL-D. Table 2 demonstrates strong scalability of CCL-D in diagnosing slow/hang anomalies across heterogeneous clusters of varying sizes. Within two months, we collected diagnostic logs from training tasks spanning tens to 4,000 GPUs to comprehensively evaluate its performance. The distribution of abnormal operations detected in Table 2 is as follows: there are 16 cases in the range [16, 128], 9 cases in (128, 512], 4 cases in (512, 1024], 4 cases in (1024, 2048], and 3 cases in (2048, 4000].

Before CCL-D integration, hang detection relied mainly on PyTorch watchdog, which only triggered after the 30-minute timeout. Locating the hang root causes across different GPU scales took an average of 47 hours per successful case. Slow anomalies depended on unpredictable customer feedback and human intuition, with an average location time of 74 hours. Furthermore, coverage of root causes was limited and relied heavily on manual expertise and coarse-grained inference, leading to inefficiency and lack of systematic precision.

With CCL-D, diagnosis becomes automated, covering both intra- and inter-node causes, and anomaly captures increase from 8 to 28. Due to our system's configuration, the system detects hang anomalies within 5 minutes and slow anomalies within 1 minute, and can infer root causes and pinpoint faulty ranks within 1 minute even at the 4,000-GPU scale. Real-world deployments confirm CCL-D as an efficient and accurate solution for large-scale model training, offering a reliable reference for addressing slow/hang anomalies.

6.3 Anomaly-Free Overhead Measurement

Diagnostic runtime overhead is evaluated from three perspectives. First, we examine system resource usage, including memory and compute cores. Second, we measure the overhead introduced at the communication operation level. Third, we assess the impact on model training in terms of efficiency and accuracy. For comparison, we select RAS and C4D as baselines, since they operate at the CCL level like CCL-D and do not interrupt model training.

6.3.1 Overhead of Tracing Framework. In this experiment, the differences across parallel strategies of model training are minimal. For clarity, we present results of BaiLing-80B under 16 to 128 GPU configurations with 3D parallel. In terms of resource consumption, RAS, C4D, and CCL-D mainly rely on host-side measurements of collective communication states, incurring negligible overhead on GPU memory and computation. For CPU resources, as shown in the right subfigure of Figure 11, CCL-D maintains a stable CPU utilization of around 0.3% per node across all scales. C4D shows a similar CPU usage trend as CCL-D. In contrast, RAS overhead increases significantly with the number of ranks, revealing poor scalability.

For host memory usage, CCL-D adopts fixed-size buffers and memory reuse, keeping the per-rank footprint stable at 1184 Bytes regardless of cluster scale. This design aligns

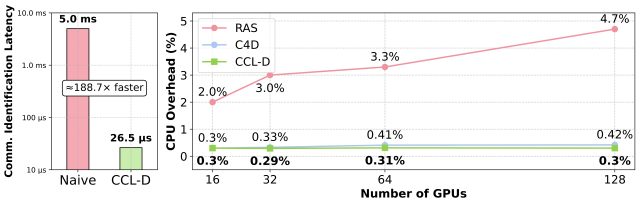


Figure 11. Communication traffic identification overhead and CPU usage per node for anomaly diagnosis at different GPU scales.

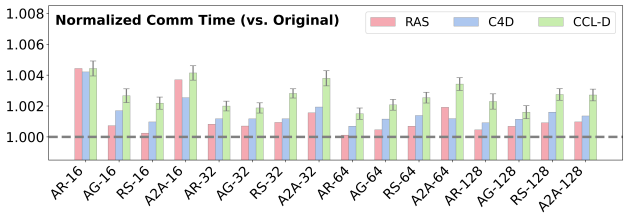


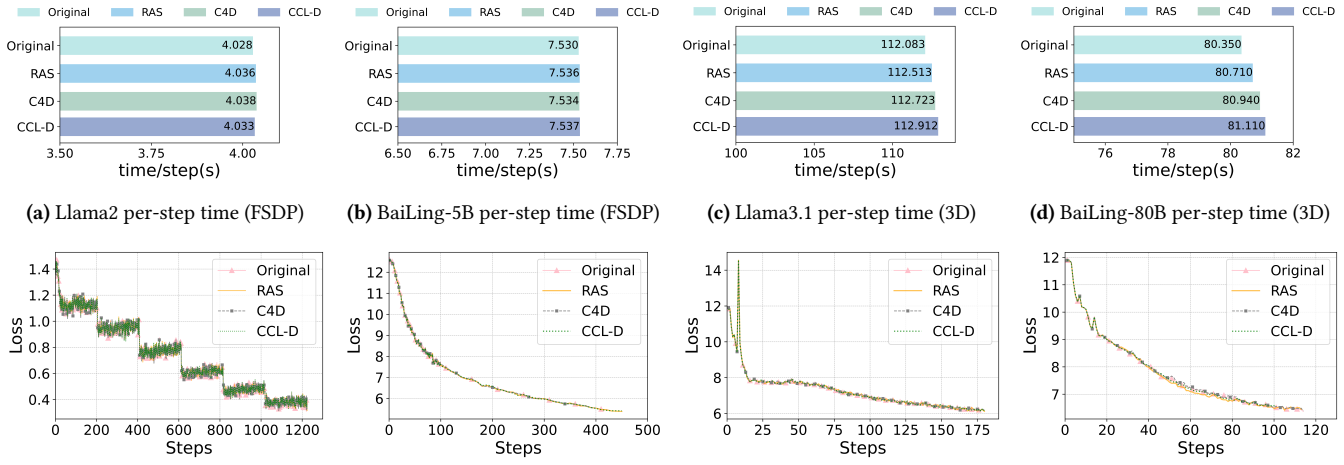
Figure 12. Comparison of normalized communication time (vs. Original) for different operations on 16–128 GPUs.

with Section 5.1, where the probing frame consists of a 32-byte shared header and a 1152-byte body divided into 8 sub-blocks, each corresponding to one Trace ID and mapped to 8 communication channels. Without using the reserved bytes, each Trace ID occupies 16 Bytes. The memory usage of RAS and C4D could not be accurately measured due to interference from training workloads. Nevertheless, RAS inherently incurs higher memory usage from constructing communicator topologies, and excessive overhead may even disrupt normal collective communication behavior.

Our diagnostic data structures not only incur negligible memory overhead, but also deliver substantial performance gains. To validate their effectiveness, we implemented a centralized naive baseline, where each communication requires a request to the identifier device. As shown in Figure 11, CCL-D reduces once identification latency from milliseconds to nanoseconds, achieving about 188× improvement.

6.3.2 Communication Operation Measurements. We evaluate four commonly used collective communication operations in large-scale model training—AllReduce, AllGather, ReduceScatter, and AlltoAll—with data sizes of 64MB, 256MB, 1GB, and 8GB. Each operation is tested 300 times, and the average is reported. Figure 12 presents the normalized communication time with diagnostics relative to the original baseline under 256MB, where the dashed line denotes the original communication. The bar-top segments of CCL-D indicate the variance across different data sizes.

RAS, C4D, and CCL-D all introduce minimal impact on communication operations, with less than 0.45% additional cost compared to orignal. This low overhead is primarily due to all systems relying on host-driven measurement, which avoids interfering with device-side communication execution. However, CCL-D incurs slightly higher overhead than



(a) Llama2 per-step time (FSDP) **(b)** BaiLing-5B per-step time (FSDP) **(c)** Llama3.1 per-step time (3D) **(d)** BaiLing-80B per-step time (3D)

(e) Llama2 loss (FSDP) **(f)** BaiLing-5B loss (FSDP) **(g)** Llama3.1 loss (3D) **(h)** BaiLing-80B loss (3D)

Figure 13. Per-step time and loss over time under different large models.

others, as it measures fine-grained metrics inside GPU kernels to ensure higher diagnostic accuracy. Although RAS achieves lower overhead by only recording operation counts, its limited anomaly coverage is unacceptable.

6.3.3 Training Efficiency and Accuracy. We test 4 models with FSDP or 3D parallelism. Llama2-7B and Llama3.1-8B are trained on 16 GPUs, BaiLing-5B on 32 GPUs, and BaiLing-80B on 48 GPUs. Figure 13 shows the impact of different diagnostic solutions on the training time and loss per step.

As shown in Figure 13 (a)–(d), for Llama2 and BaiLing-5B, CCL-D exhibits insensitivity to model types under the same parallel strategy, with maximum overhead as low as 0.12%. However, compared to FSDP, 3D parallelism offers finer granularity for both training data and model partitioning, resulting in more communicators and higher communication frequency. This lead to 0.74% and 0.95% overhead for Llama3.1 and BaiLing-80B. In contrast, RAS, due to its simplistic metric collection, shows lower sensitivity across models and parallel modes, with a maximum overhead of 0.45%. Nevertheless, reducing runtime overhead at the cost of diagnostic accuracy is not a desirable trade-off.

Figure 13 (e)-(h) show the training accuracy of different large models. After applying the diagnostic systems, the downward trend of the loss function remains consistent with that of the original training. This is expected, as RAS, C4D, and CCL-D did not modify the model algorithm or training data, thus having no impact on the loss function.

7 Related Work

Diagnostic systems are vital for ensuring hardware stability and correct software execution in large-scale training, and have been widely studied across academia and industry.

Training-centric anomaly diagnosis systems. MegaScale [19] scales LLM training to over 10,000 GPUs, employing heartbeat checks, lightweight self-tests, and CUDA event analysis to enhance cluster stability, but its diagnosis remains at the job/node granularity and does not extend into communication kernels. Minder [10] leverages metric similarity and continuity for node-level anomaly identification, showing effectiveness in large production clusters but remaining machine-centric. Dynolog [30] integrates PyTorch Profiler into production to capture coarse-grained GPU kernel traces for online monitoring. Superbench [54] provides a comprehensive benchmarking suite covering operator, bandwidth, and latency metrics, yet focuses on pre-deployment checks rather than runtime diagnosis. SkeletonHunter [27] exploits tensor skeleton features to detect faulty network components, demonstrating strong results under multi-model training tasks. Overall, these approaches enhance observability at system or node granularity but fall short of probing communication kernels, limiting their coverage of slow/hang.

General network failure diagnosis systems. CommScribe [1, 48] monitors collective communication using transfer matrices of size and frequency to identify bottlenecks and risks. NetBouncer [42] combines IP-tunnel probing with inference algorithms to detect link and device failures via machine learning. Hostping [26] and Justitia [55] monitor host, bus, and NIC states through hardware sensors and loopback tests, diagnosing anomalies in latency and bandwidth. In contrast, CCL-D operates at the CCL level with lightweight tracing, enabling precise slow/hang diagnosis without external hardware dependency. Moreover, CCL-D is orthogonal to these methods: while network/system-level tools capture infrastructure issues, CCL-D focuses on kernel-level communication states. Their combination offers a more complete diagnostic coverage for large-scale training.

8 Conclusion

Slow/Hang anomalies in CCL frequently occur and are difficult to diagnose during large-scale training. In this paper,

we introduce CCL-D, which carefully designs a set of high-precision metrics for runtime probing, traces each collective communication in a decentralized manner, and measures the communication with low overhead. Then, it rapidly analyzes the metrics and locates specific faulty ranks using an efficient decision algorithm. Experiments show that CCL-D achieves superior coverage and accuracy, diagnosing slow/hang anomalies in large-scale training within 6 minutes.

Acknowledgments

We would like to acknowledge support from the National Key Research and Development Program of China (Grant No. 2025YFB3003702), the National Natural Science Foundation of China (Grant Nos. 62032023 and T2125013), and the Ant Group SCT. The AI-driven experiments, simulations and model training were performed on the robotic AI-Scientist platform of Chinese Academy of Sciences.

References

[1] Palwisha Akhtar, Erhan Tezcan, Fareed Mohammad Qararyah, and Didem Unat. 2020. ComScribe: identifying intra-node GPU communication. In *International Symposium on Benchmarking, Measuring and Optimization*. Springer, 157–174.

[2] AMD. 2025. RCCL: ROCm Communication Collectives Library. https://github.com/ROCm/rccl. Accessed August 25, 2025.

[3] BigScience. 2025. BLOOM 176B Training Log. https://github.com/bigscience-workshop/bigscience/blob/master/train/tr11-176B-ml/chronicles.md. Accessed August 25, 2025.

[4] Sanghun Cho, Hyojun Son, and John Kim. 2023. Logical/physical topology-aware collective communication in deep learning training. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 56–68.

[5] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. NVIDIA a100 tensor core gpu: Performance and innovation. *IEEE Micro* 41, 2 (2021), 29–35.

[6] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 1–22.

[7] Can Cui, Yunsheng Ma, Xu Cao, Wenqian Ye, Yang Zhou, Kaizhao Liang, Jintai Chen, Juanwu Lu, Zichong Yang, Kuei-Da Liao, et al. 2024. A survey on multimodal large language models for autonomous driving. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 958–979.

[8] Weihao Cui, Ji Zhang, Han Zhao, Chao Liu, Wenhao Zhang, Jian Sha, Quan Chen, Bingsheng He, and Minyi Guo. 2025. XPUTimer: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale. *arXiv preprint arXiv:2502.05413* (2025).

[9] Huangliang Dai, Shixun Wu, Jiajun Huang, Zizhe Jian, Yue Zhu, Haiyang Hu, and Zizhong Chen. 2025. FT-Transformer: Resilient and reliable transformer with end-to-end fault tolerant attention. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1085–1098.

[10] Yangtao Deng, Xiang Shi, Zhuo Jiang, Xingjian Zhang, Lei Zhang, Zhang Zhang, Bo Li, Zuquan Song, Hang Zhu, Gaohong Liu, et al. 2025. Minder: Faulty machine detection for large-scale distributed model training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 505–521.

[11] Jianbo Dong, Bin Luo, Jun Zhang, Pengcheng Zhang, Fei Feng, Yikai Zhu, Ang Liu, Zian Chen, Yi Shi, Hairong Jiao, et al. 2025. Enhancing Large-Scale AI Training Efficiency: The C4 Solution for Real-Time Anomaly Detection and Communication Optimization. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1246–1258.

[12] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[13] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. 2022. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 929–943.

[14] Yanjie Gao, Jiyu Luo, Haoxiang Lin, Hongyu Zhang, Ming Wu, and Mao Yang. 2025. dl^2: Detecting Communication Deadlocks in Deep Learning Jobs. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 27–38.

[15] Jiayi Huang, Pritam Majumder, Sungkeun Kim, Abdullah Muzahid, Ki Hwan Yum, and Eun Jung Kim. 2021. Communication algorithm-architecture co-design for distributed deep learning. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 181–194.

[16] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).

[17] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 947–960.

[18] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 463–479.

[19] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 745–760.

[20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[21] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT*, Vol. 1. Minneapolis, Minnesota.

[22] Hongbo Li, Zizhong Chen, and Rajiv Gupta. 2017. Parastack: Efficient hang detection for mpi programs at large scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12.

[23] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704* (2020).

[24] Wenshuo Li, Xinghao Chen, Han Shu, Yehui Tang, and Yunhe Wang. 2024. ExCP: Extreme LLM Checkpoint Compression via Weight-Momentum Joint Shrinking. *arXiv preprint arXiv:2406.11257* (2024).

[25] Yan-Bo Lin, Yi-Lin Sung, Jie Lei, Mohit Bansal, and Gedas Bertasius. 2023. Vision transformers are parameter-efficient audio-visual learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2299–2309.

[26] Kefei Liu, Zhuo Jiang, Jiao Zhang, Haoran Wei, Xiaolong Zhong, Lizhuang Tan, Tian Pan, and Tao Huang. 2023. Hostping: Diagnosing intra-host network bottlenecks in RDMA servers. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 15–29.

[27] Wei Liu, Kun Qian, Zhenhua Li, Tianyin Xu, Yunhao Liu, Weicheng Wang, Yun Zhang, Jiakang Li, Shuhong Zhu, Xue Li, et al. 2025. SkeletonHunter: Diagnosing and Localizing Network Failures in Containerized Large Model Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 527–540.

[28] Keith Marzullo and Susan Owicki. 1983. Maintaining the time in a distributed system. In *Proceedings of the second annual ACM symposium on Principles of distributed computing*. 295–305.

[29] Avinash Maurya, Robert Underwood, M Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. Datastates-llm: Lazy asynchronous checkpointing for large language models. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. 227–239.

[30] Meta. 2025. Dynolog : a telemetry daemon for performance monitoring and tracing. https://github.com/facebookincubator/dynolog. Accessed August 25, 2025.

[31] Meta. 2025. OPT 175B Training Log. https://github.com/facebookresearch/metaseq/blob/main/projects/OPT/chronicles/OPT175B_Logbook.pdf. Accessed August 25, 2025.

[32] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.

[33] Thanh-Dat Nguyen, Haoye Tian, Bach Le, Patanamon Thongtanunam, and Shane McIntosh. 2025. A Systematic Survey on Debugging Techniques for Machine Learning Systems. *arXiv preprint arXiv:2503.03158* (2025).

[34] NVIDIA. 2025. Collective Communication Protocol. https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html. Accessed August 25, 2025.

[35] NVIDIA. 2025. NCCL RAS. https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/ras.html. Accessed August 25, 2025.

[36] NVIDIA. 2025. Nccl-tests. https://github.com/NVIDIA/nccl-tests. Accessed August 25, 2025.

[37] NVIDIA. 2025. NVIDIA Nsight Compute. https://docs.nvidia.com/nsight-compute/NsightCompute/index.html. Accessed August 25, 2025.

[38] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. 2024. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557* (2024).

[39] Sreeram Potluri, Khaled Hamidouche, Akshay Venkatesh, Devendar Bureddy, and Dhabaleswar K Panda. 2013. Efficient inter-node MPI communication using GPUDirect RDMA for InfiniBand clusters with NVIDIA GPUs. In *2013 42nd International Conference on Parallel Processing*. IEEE, 80–89.

[40] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[41] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[42] Cheng Tan, Ze Jin, Chuanxiong Guo, Tianrong Zhang, Haitao Wu, Karl Deng, Dongming Bi, and Dong Xiang. 2019. NetBouncer: Active device and link failure localization in data center networks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 599–614.

[43] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.

[44] DLRover Team. 2025. DLRover. https://github.com/intelligent-machine-learning/dlrover. Accessed August 25, 2025.

[45] Ling Team, Binwei Zeng, Chao Huang, Chao Zhang, Changxin Tian, Cong Chen, Dingnan Jin, Feng Yu, Feng Zhu, Feng Yuan, et al. 2025. Every FLOP Counts: Scaling a 300B Mixture-of-Experts LING LLM without Premium GPUs. *arXiv preprint arXiv:2503.05139* (2025).

[46] Torch Team. 2025. Pytorch Watchdog. https://pytorch.org/docs/stable/torch_nccl_environment_variables.html. Accessed August 25, 2025.

[47] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[48] Didem Unat. 2022. Monitoring Collective Communication Among GPUs. In *Euro-Par 2021: Parallel Processing Workshops: Euro-Par 2021 International Workshops, Lisbon, Portugal, August 30-31, 2021, Revised Selected Papers*, Vol. 13098. Springer Nature, 41.

[49] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[50] Boxiang Wang, Qifan Xu, Zhengda Bian, and Yang You. 2022. Tesseract: Parallelize the tensor parallelism efficiently. In *Proceedings of the 51st International Conference on Parallel Processing*. 1–11.

[51] Yuxin Wang, Shaohuai Shi, Xin He, Zhenheng Tang, Xinglin Pan, Yang Zheng, Xiaoyu Wu, Amelie Chi Zhou, Bingsheng He, and Xiaowen Chu. 2023. Reliable and efficient in-memory fault tolerance of large language model pretraining. *arXiv preprint arXiv:2310.12670* (2023).

[52] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, TS Eugene Ng, and Yida Wang. 2023. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 364–381.

[53] Tianyuan Wu, Wei Wang, Yinghao Yu, Siran Yang, Wenchao Wu, Qinkai Duan, Guodong Yang, Jiamang Wang, Lin Qu, and Liping Zhang. 2025. GREYHOUND: Hunting Fail-Slows in Hybrid-Parallel Training at Scale. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 731–747.

[54] Yifan Xiong, Yuting Jiang, Ziyue Yang, Lei Qu, Guoshuai Zhao, Shuguang Liu, Dong Zhong, Boris Pinzur, Jie Zhang, Yang Wang, et al. 2024. SuperBench: Improving Cloud AI Infrastructure Reliability with Proactive Validation. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 835–850.

[55] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. 2022. Justitia: Software Multi-Tenancy in Hardware Kernel-Bypass Networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 1307–1326.

[56] Hairui Zhao, Hongliang Li, Qi Tian, Jie Wu, Meng Zhang, Zhewen Xu, Xiang Li, and Haixiao Xu. 2025. ArrayPipe: Introducing Job-Array Pipeline Parallelism for High Throughput Model Exploration. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.

[57] Hairui Zhao, Qi Tian, Hongliang Li, and Zizhong Chen. 2025. {FlexPipe}: Maximizing training efficiency for transformer-based models with {Variable-Length} inputs. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 143–159.

[58] Jingyuan Zhao, Wenyi Zhao, Bo Deng, Zhenghong Wang, Feng Zhang, Wenxiang Zheng, Wanke Cao, Jinrui Nan, Yubo Lian, and Andrew F Burke. 2024. Autonomous driving system: A comprehensive survey.

Expert Systems with Applications 242 (2024), 122836.

[59] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).

Received 2025-08-23; accepted 2025-11-10